

Sorting Algorithms

Number of compares of common sorting algorithms

ALGORITHM	CODE	IN PLACE	STABLE	BEST	AVERAGE	WORST	REMARKS
selection sort	Selection.java 	✓		$\frac{1}{2} n^2$	$\frac{1}{2} n^2$	$\frac{1}{2} n^2$	n exchanges; quadratic in best case
insertion sort	Insertion.java 	✓	✓	n	$\frac{1}{4} n^2$	$\frac{1}{2} n^2$	use for small or partially-sorted arrays
bubble sort	Bubble.java 	✓	✓	n	$\frac{1}{2} n^2$	$\frac{1}{2} n^2$	rarely useful; dominated by insertion sort
shellsort	Shell.java 	✓		$n \log_3 n$	unknown	$c n^{3/2}$	tight code; subquadratic
mergesort	Merge.java 		✓	$\frac{1}{2} n \log_2 n$	$n \log_2 n$	$n \log_2 n$	$n \log n$ guarantee; stable
quicksort	Quick.java 	✓		$n \log_2 n$	$2 n \ln n$	$\frac{1}{2} n^2$	$n \log n$ probabilistic guarantee; fastest in practice
heapsort	Heap.java 	✓		$n^\dagger$	$2 n \log_2 n$	$2 n \log_2 n$	$n \log n$ guarantee; in place

[†] $n \log_2 n$ if all keys are distinct

It includes leading constants but ignores lower-order terms

Selection Sort and Bubble Sort

Selection Sort

Idea:

- Finds the minimum element in the unsorted part of the array.
- Swaps it with the first element of the unsorted part.
- Repeats this process for the remaining unsorted part.
- Unstable Sort

8
5
2
6
9
3
1
4
0
7

Bubble Sort

Idea:

- Compares adjacent elements and swaps them if they are in the wrong order.
- Repeats this process until no more swaps are needed.
- Stable Sort

6 5 3 1 8 7 2 4

Images source wikipedia

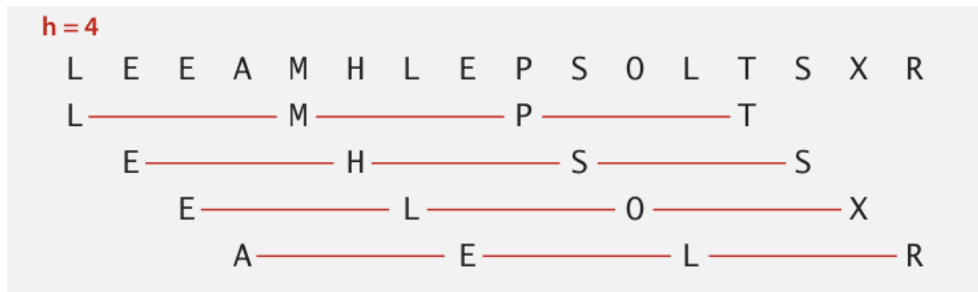
Insertion Sort

```
def insertion_sort(arr):  
    for k in range(1, len(arr)): # from 1 to n-1  
        cur = arr[k] # current element to be inserted  
        j = k # find correct index j for current  
        while j > 0 and arr[j-1] > cur: # element A[j-1] must be after current  
            arr[j] = arr[j-1]  
            j -= 1  
        arr[j] = cur #final position for the current value  
    return arr
```

Shell sort

Idea: Move entries more than one position at a time by h-sorting the array.

An h-sorted array is nothing but h-interleaved sorted sub-sequences



Shellsort: h-sort array for decreasing sequence of values of h

Shell sort - example

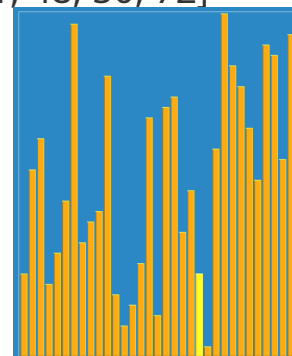
Input: Arr = [72, 50, 48, 41, 33, 23, 20, 19, 18, 16, 14, 13, 11, 9, 6, 5, 1]

Outputs:

For h = 13, [9, 6, 5, 1, 33, 23, 20, 19, 18, 16, 14, 13, 11, 72, 50, 48, 41]

For h = 4, [9, 6, 5, 1, 11, 16, 14, 13, 18, 23, 20, 19, 33, 72, 50, 48, 41]

For h = 1, [1, 5, 6, 9, 11, 13, 14, 16, 18, 19, 20, 23, 33, 41, 48, 50, 72]



Shell sort

```
def shell_sort(arr):
    n_len = len(arr)
    # Compute the largest possible h for the input
    h = 1
    while h < n_len // 3:
        h = 3 * h + 1 # Knuth's sequence

    # Perform Shell Sort using Knuth's sequence
    while h > 0:
        for i in range(h, n_len):
            # Save the element and its index for insertion
            curr = arr[i]
            j = i

            # Shift elements of the sorted subarray
            while j >= h and arr[j - h] > curr:
                arr[j] = arr[j - h]
                j -= h

            # Place the 'curr' in its correct position
            arr[j] = curr

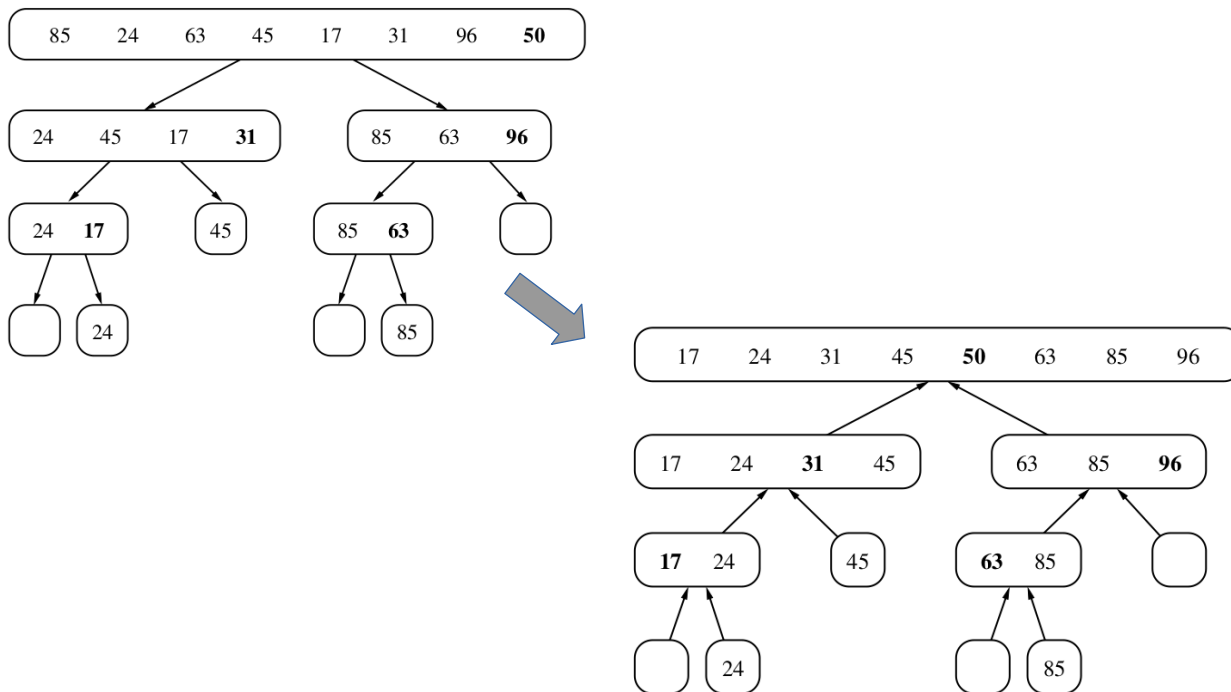
        # Reduce the h according to Knuth's sequence
        h //= 3
    return arr
```

```
def insertion_sort(arr):
    for k in range(1, len(arr)): # for each element
        cur = arr[k] # current element to be inserted
        j = k # find correct index j for current element
        while j > 0 and arr[j-1] > cur:
            arr[j] = arr[j-1]
            j -= 1
        arr[j] = cur
    return arr
```

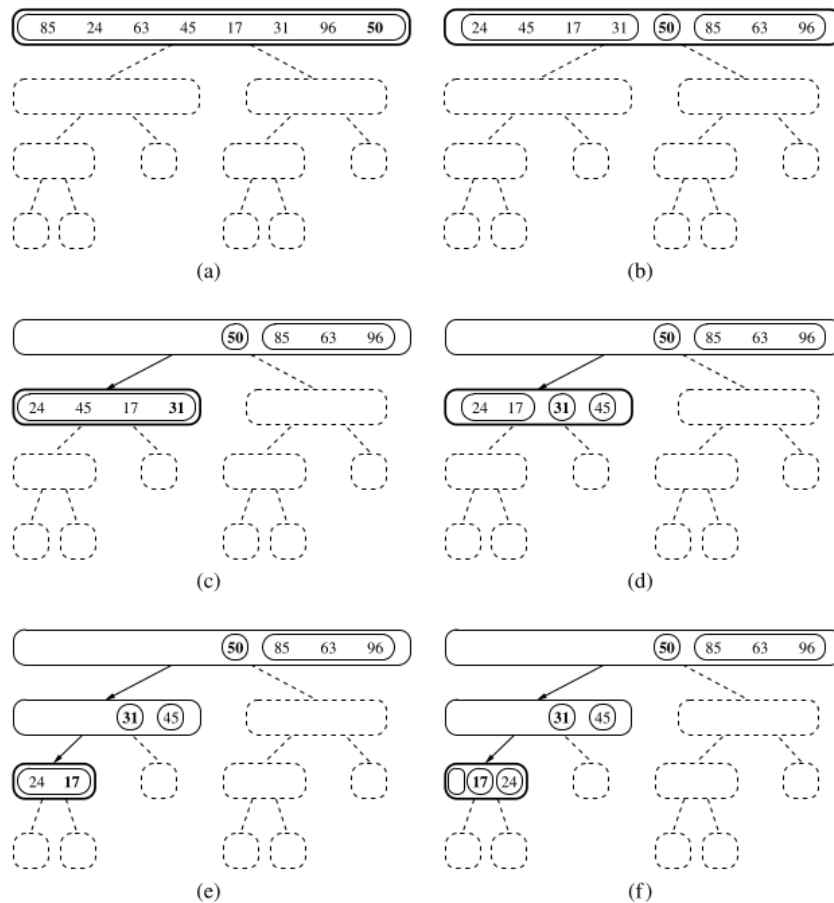
Comparison: Insertion Sort vs Shell Sort

Features	Insertion Sort	Shell Sort
Approach	Sorts one element at a time, iteratively builds the sorted array.	Uses hops to sort subarrays, progressively reduces hops to optimize sorting.
Time Complexity	$O(n^2)$ - worst and average cases. $O(n)$ - the best case for nearly sorted data.	Depends on the gap sequence: $O(n^2)$ worst case for basic gaps, can be as good as $O(n \log^3 n)$ in best case.
Space Complexity	$O(1)$ (in-place sorting).	$O(1)$ (in-place sorting).
Stability	Stable (maintains relative order of equal elements).	Can be unstable (swapping may alter order of equal elements).
Best Use Case	Efficient for small datasets or nearly sorted arrays.	Better for larger datasets where gaps reduce the number of comparisons.
Efficiency	Slower for larger datasets due to excessive shifting of elements.	Faster than insertion sort for larger datasets due to reduced inversions early on.

Quicksort



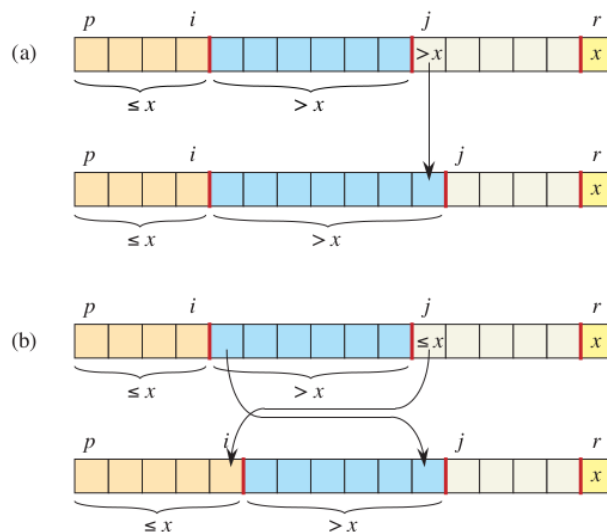
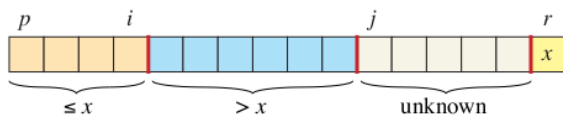
Order of
function
calls (only few
are shown here)



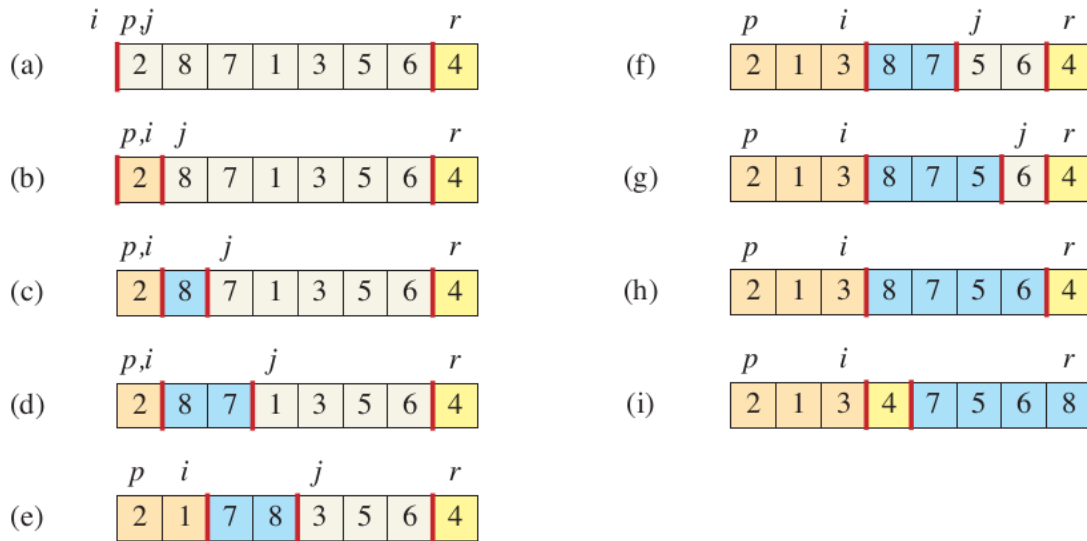
Quicksort – A divide & conquer approach

1. Divide by partitioning the array into two (possibly empty) subarrays based on a pivot. Compute the index of the pivot as part of this partitioning procedure.
2. Conquer by calling quicksort recursively to sort each of the subarrays
3. Combine by doing nothing: because the two subarrays are already sorted, no work is needed to combine them.

Quicksort – more details



Quicksort – more details



Quicksort – Pseudo code

QUICKSORT(A, p, r)

```

1  if  $p < r$ 
2      // Partition the subarray around the pivot, which ends up in  $A[q]$ .
3       $q = \text{PARTITION}(A, p, r)$ 
4      QUICKSORT( $A, p, q - 1$ ) // recursively sort the low side
5      QUICKSORT( $A, q + 1, r$ ) // recursively sort the high side

```

PARTITION(A, p, r)

```

1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot

```

Quicksort – perhaps the most concise code you can ever have

```
9  arr = [3,4,5,33,13,1,2,12,8,9,7,6]
10
11  def qsort(arr):
12
13      if(len(arr) <= 1):
14          return arr
15
16      piv = arr[0]
17
18      sortarr = qsort([i for i in arr[1:] if i <= piv]) + \
19      [piv] + qsort([i for i in arr[1:] if i > piv])
20
21      return sortarr
22
23
24  print(qsort(arr))
```

Performance of quicksort

Worst-case partitioning: $T(n) = T(n - 1) + T(0) + \Theta(n)$
 $= T(n - 1) + \Theta(n)$.

Best-case partitioning:

produces two sub problems, each of size no more than $n/2$,

$$T(n) = 2T(n/2) + \Theta(n) .$$

Randomised quicksort

RANDOMIZED-PARTITION(A, p, r)

```
1   $i = \text{RANDOM}(p, r)$ 
2  exchange  $A[r]$  with  $A[i]$ 
3  return PARTITION( $A, p, r$ )
```

RANDOMIZED-QUICKSORT(A, p, r)

```
1  if  $p < r$ 
2       $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ 
3      RANDOMIZED-QUICKSORT( $A, p, q - 1$ )
4      RANDOMIZED-QUICKSORT( $A, q + 1, r$ )
```

Lower Bound for Sorting

The running time of any comparison-based algorithm for sorting an n -element sequence is $\Omega(n \log n)$ in the worst case.

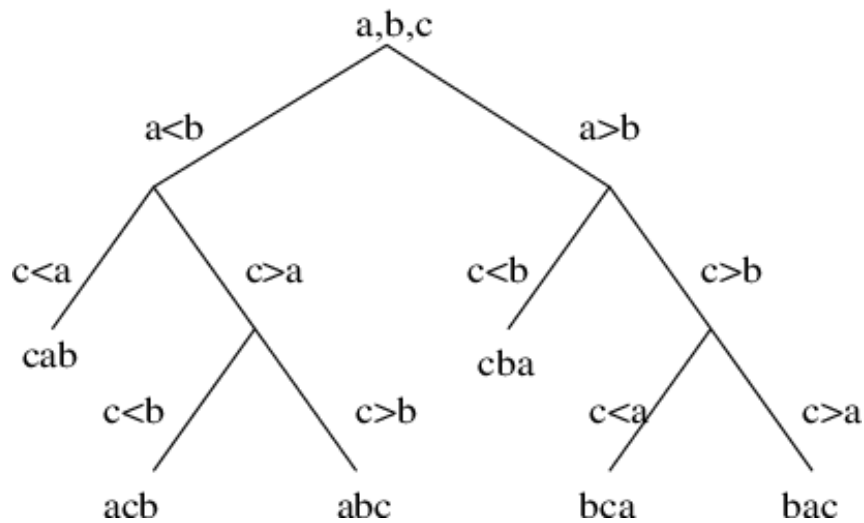
W.K.T, a sorting algorithm essentially outputs a permutation of the input

Let $S = (x_0, x_1, \dots, x_{n-1})$ be an input sequence to be sorted

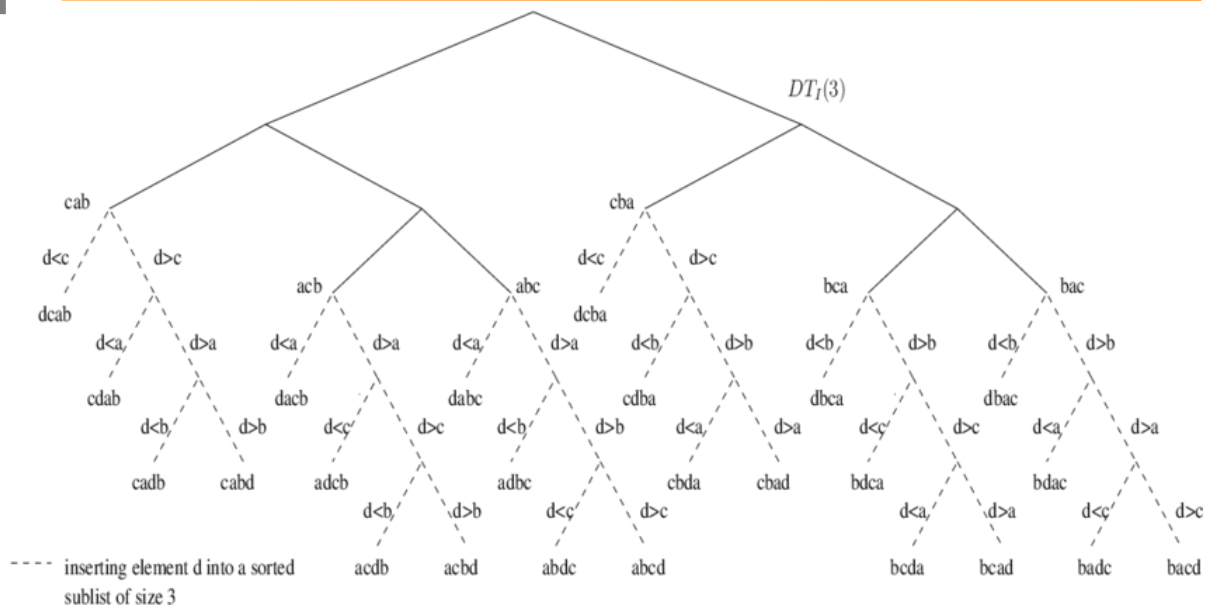
Key observations:

(i) there are $n!$ different possible permutations the algorithm might output

Lower Bound for Sorting



Lower Bound for Sorting



Lower Bound for Sorting

(ii) for each of these permutations, there exists an input for which that permutation is the only correct answer.

e.g., $[x_3, x_2, x_0, x_1]$ is the only correct answer for sorting the input $[7, 8, 4, 3]$

ie. we can fix some set of $n!$ inputs (e.g., all orderings of $\{1, 2, \dots, n\}$), one for each of the $n!$ output permutations.

Lower Bound for Sorting

Each external node in decision tree T must be associated with one permutation of S . Moreover, each permutation of S must result in a different external node of T .

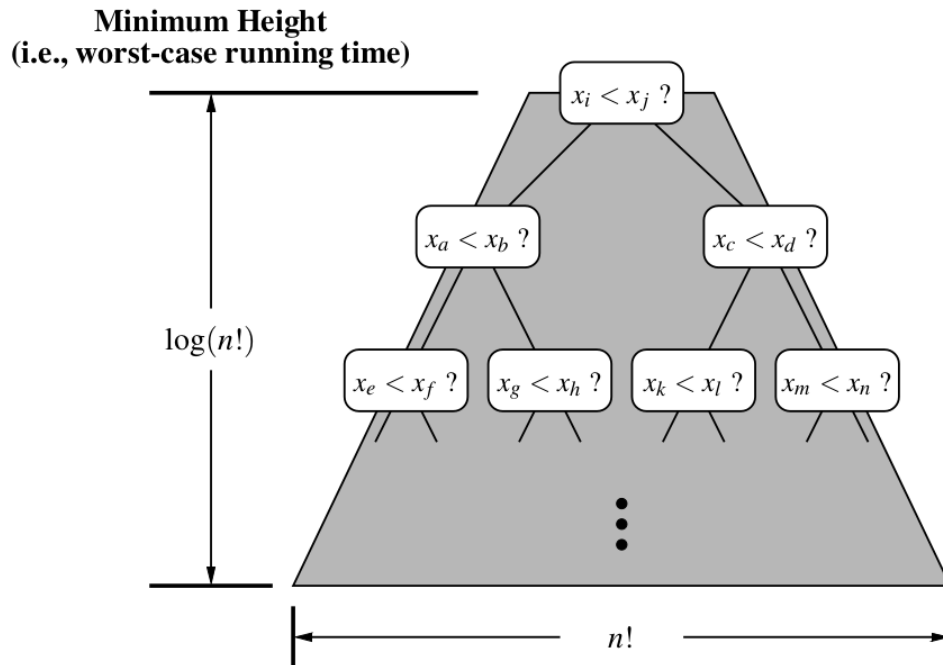
Thus, T must have at least $n!$ external nodes. For a proper binary tree having n nodes, we know that $\log(n + 1) - 1 \leq h \leq (n - 1)/2$, the height of T is at least $\log(n!)$

$$\log(n!) \geq \log \left(\left(\frac{n}{2} \right)^{\frac{n}{2}} \right) = \frac{n}{2} \log \frac{n}{2},$$

which is $\Omega(n \log n)$

Since there are at least $n/2$ terms that are greater than or equal to $n/2$ in the product $n!$

Lower Bound for Sorting



Non-comparison-based sorting algorithms

Non-comparison-based sorting algorithms such as Radix Sort and Bucket Sort don't rely on directly comparing elements to determine their order.

Bucket Sort: A sequence S of n entries whose keys are integers in the range $[0, N - 1]$, for some integer $N \geq 2$, can be sorted in $O(n + N)$ time.

Both Bucket and Radix sorting methods are stable

Bucket Sort

Algorithm bucketSort(S):

Input: Sequence S of entries with integer keys in the range $[0, N - 1]$

Output: Sequence S sorted in nondecreasing order of the keys

let B be an array of N sequences, each of which is initially empty

for each entry e in S **do**

 k = the key of e

 remove e from S and insert it at the end of bucket (sequence) B[k]

for i = 0 to N-1 **do**

for each entry e in sequence B[i] **do**

 remove e from B[i] and insert it at the end of S

Radix Sort - example

How to sort a sequence

$S = ((3, 3), (1, 5), (2, 5), (1, 2), (2, 3), (1, 7), (3, 2), (2, 2)).$

Ans:

Stably sort the 2nd component followed by the 1st (ie. start from rightmost and traverse to left side)

$S_2 = ((1, 2), (3, 2), (2, 2), (3, 3), (2, 3), (1, 5), (2, 5), (1, 7)).$

$S_{2,1} = ((1, 2), (1, 5), (1, 7), (2, 2), (2, 3), (2, 5), (3, 2), (3, 3)),$

Radix Sort - another example

170,045,075,090,802,024,002,066

First we identify each right-most (least significant) digit:

170, 045, 075, 090, 802, 024, 002, 066

We sort the list according to that digit, preserving order within that group:

170, 090, 802, 002, 024, 045, 075, 066

Then we identify the next digit, moving leftwards:

170, 090, 802, 002, 024, 045, 075, 066

We sort the list according to that digit, preserving order within that group:

802, 002, 024, 045, 066, 170, 075, 090

Then we identify the next digit, moving leftwards:

802, 002, 024, 045, 066, 170, 075, 090

We sort the list according to that digit, preserving order within that group:

002, 024, 045, 066, 075, 090, 170, 802,

Radix Sort - another example

Proposition 12.6: *Let S be a sequence of n key-value pairs, each of which has a key $(k_1, k_2, \dots, k_d)$, where k_i is an integer in the range $[0, N - 1]$ for some integer $N \geq 2$. We can sort S lexicographically in time $O(d(n + N))$ using radix-sort.*

- Like bucket-sort, radix sort is efficient when the range N of values for the keys is small compared to the sequence size n , say $N = O(n)$ or $N = O(n \log n)$.
- However, its performance deteriorates as N grows compared to n .

Timsort – a relatively new kid on the block

1. Timsort is a hybrid, stable sorting algorithm, derived from merge sort and insertion sort
2. Implemented by Tim Peters in 2002 for use in the Python programming language
3. The algorithm finds sub-sequences of the data that are already ordered (runs) and uses them to sort the remainder more efficiently.
4. provides good average-case time complexity of $O(n \log n)$, similar to Merge Sort. It also performs well for partially sorted data, making it a versatile choice for Python's default sorting mechanism.

Exercise Problems

Show that the best-case running time of quick-sort on a sequence of size n with distinct elements is $\Omega(n \log n)$.

Of the $n!$ possible inputs to a given comparison-based sorting algorithm, what is the absolute maximum number of inputs that could be correctly sorted with just n comparisons?

Jonathan has a comparison-based sorting algorithm that sorts the first k elements of a sequence of size n in $O(n)$ time. Give a big-Oh characterization of the biggest that k can be?

Exercise Problems

Is the bucket-sort algorithm in-place? Why or why not?

Suppose S is a sequence of n values, each equal to 0 or 1. How long will it take to sort S with the merge-sort algorithm? What about quick-sort?

Suppose S is a sequence of n values, each equal to 0 or 1. How long will it take to sort S stably with the bucket-sort algorithm?

Give an example input list that requires merge-sort and heap-sort to take $O(n \log n)$ time to sort, but insertion-sort runs in $O(n)$ time. What if you reverse this list?

Exercise Problems

Assume that a mergesort algorithm in the worst case takes 30 seconds for an input of size 64. Which of the following most closely approximates the maximum input size of a problem that can be solved in 6 minutes?

- ☐ 256
- ☐ 512
- ☐ 1024
- ☐ 2048



References

Data Structures and Algorithms in Python

Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser

Introduction to Algorithms

Leiserson, Stein, Rivest, Cormen

Algorithms, 4th Edition

Robert Sedgewick and Kevin Wayne

Few Images from the internet



Quickselect

- An algorithm for finding the k -th smallest element in an unordered list.
- Quickselect is efficient for selecting a single element, especially in scenarios where sorting the entire list is unnecessary.
- The average-case complexity is $O(n)$ since the algorithm processes only one side of the partition in each step as opposed to both sides in Quicksort